

ESTRUCTURAS DE DATOS - 2026-2 Implementación y Análisis de Complejidad de Listas, Pilas y Colas en Java

Estudiante: Marco Antonio García Villasmil Código: 1094050756 Profesor: David Herrera Monitora: Angela Camila Siabato Londoño

Repositorio: <https://github.com/marcogarciavillasmil/lab-listas-pilas-colas>

1. Objetivo del Taller

Acá tocaba armar en Java la estructura List con listas enlazadas, en sus cuatro variantes: simple sin cola, simple con cola, doble sin cola y doble con cola. También había que hacer Stack y Queue, cada una en dos versiones (arreglo dinámico y arreglo circular), para un total de 8 implementaciones. Con todo compilando y funcionando, lo que realmente importa del taller es medir cuánto se demora cada método según el tamaño de la entrada, comparar eso contra lo que predice la teoría en notación Big-O, y sacar conclusiones de cuál implementación conviene según el caso.

2. Implementación

List (Listas Enlazadas)

Para esta parte armé una interfaz común, MyList, con los 8 métodos que pide el enunciado: pushFront, pushBack, popFront, popBack, find, erase, addBefore y addAfter (también agregué topFront y topBack, que uso más adelante en la sección 7). Las 4 variantes de abajo implementan exactamente lo mismo, solo que manejan los punteros de forma distinta por dentro.

- **SinglyLinkedListNoTail:** La versión más simple de las cuatro, solo guarda un puntero a head. Cualquier operación que necesite llegar al final (pushBack, popBack, topBack) o encontrar el nodo anterior a otro (erase, addBefore) termina recorriendo la lista completa, porque no hay una forma más corta de llegar hasta ahí.
- **SinglyLinkedListWithTail:** Acá agregué un puntero tail, que resuelve pushBack y topBack de una (ya no toca recorrer), pero popBack sigue quedando mal: sin puntero al nodo anterior, para actualizar quién queda como nuevo tail toca recorrer igual desde el head.
- **DoublyLinkedListNoTail:** Cada nodo tiene ahora un puntero prev además del next, así que dado cualquier nodo ya se puede hacer erase, addBefore y addAfter en $O(1)$. Lo que queda pendiente: al no tener tail, llegar hasta el último nodo (pushBack, popBack, topBack) sigue costando $O(n)$.
- **DoublyLinkedListWithTail:** Esta junta las dos cosas (head, tail, prev y next en cada nodo) y es la única que queda en $O(1)$ en casi todo. La excepción es find, que al no haber indexación por posición no tiene otra opción que recorrer.

Algo que decidí desde el principio: find() no devuelve un booleano ni un índice, devuelve directamente la referencia al Node. La idea es no buscar el elemento dos veces: si solo dijera "sí está" o "no está", borrarlo o insertar algo al lado después tocaría recorrer la lista otra vez para ubicarlo, otro $O(n)$ de paso. Al devolver el Node, ese segundo paso queda en pura manipulación de punteros. Por eso en la tabla de 4.1 las complejidades de erase, addBefore y addAfter asumen que ya se tiene el Node en la mano (el que devolvió find()); no incluyen el costo de buscarlo, que en las 4 variantes sigue siendo $O(n)$ porque ninguna tiene indexación directa.

MyStack y MyQueue

Para estas dos estructuras terminé probando dos estrategias distintas de arreglo:

- **Arreglo Dinámico (ArrayStack / ArrayQueue):** Es un arreglo interno que se duplica cada vez que se llena, lo mismo que hace ArrayList por dentro. En el Stack esto basta para que push y pop queden en $O(1)$ amortizado, porque las dos operan sobre el tope. Donde se complica es en el Queue: enqueue mete el elemento al final sin drama, pero dequeue tiene que sacar la posición 0 y correr todos los elementos un puesto hacia atrás, lo que deja esa operación en $O(n)$.
- **Arreglo Circular (CircularArrayStack / CircularArrayQueue):** Acá cambié el arreglo plano por uno circular: guardo un índice de inicio y calculo las posiciones módulo la capacidad. Para el Stack casi no cambia nada, porque sigue operando en un solo extremo; para el Queue sí hace la diferencia: dequeue deja de mover elementos y solo avanza el índice de inicio, quedando en $O(1)$. Cuando el arreglo se llena y hay que agrandarlo, copio los elementos en su orden lógico desde ese índice y lo reinicio en 0 en el arreglo nuevo.

Para el crecimiento del arreglo (tanto en Stack como en Queue) escogí duplicar la capacidad cuando se llena (capacidad $\ast = 2$, arrancando en 16). Una inserción puntual puede salir cara, porque hay que copiar los n elementos al arreglo nuevo, pero eso solo pasa en las inserciones 16, 32, 64, 128... Si se suman todas las copias que se hacen al meter n elementos queda $16 + 32 + 64 + \dots + n/2 + n$, que es menor que $2n$, así que el costo total es $O(n)$ y cada inserción sale en $O(1)$ amortizado. Si en cambio el arreglo creciera sumando una cantidad fija k , se copiaría n/k veces y el total sería $O(n^2/k)$, o sea $O(n)$ por inserción en promedio, por eso descarté esa opción. Para ver esto en la práctica, en StackBenchmark también medí el tiempo de armar la pila desde vacía con n push (incluyendo todos los redimensionamientos) dividido entre n ; el resultado está en la sección 6.2.

3. Metodología de Medición

Para medir usé `System.nanoTime()` y reporté todo en microsegundos. Elegí `.nanoTime` y no `currentTimeMillis` porque los métodos $O(1)$ tardan decenas de nanosegundos y en milisegundos todo se redondea a 0; `.nanoTime` tiene la resolución que hace falta para ver la diferencia entre $O(1)$ y $O(n)$. Al graficar paso los nanosegundos a microsegundos, porque los números en ns llegan a millones en los métodos $O(n)$ y se pierde la comparación, mientras que en μs quedan en un rango legible. La medición está separada de la graficación: Java solo mide y escribe los csv en `results/`, y la gráfica la hace aparte un script de Python leyendo esos archivos.

Para cada tamaño n armé la estructura con n elementos usando un método $O(1)$ y mido el método. Como una operación $O(1)$ dura apenas unas decenas de nanosegundos, que es parecido al costo de llamar a `.nanoTime`, medirla una por una da mucho ruido. Por eso en los métodos de los extremos (`pushFront`, `popFront`, `topFront` y, en Stack y Queue, `push`, `pop`, `peek`, `enqueue`, `dequeue` y `front`) tomo el tiempo de un lote de 1000 operaciones con un solo par de llamadas a `.nanoTime` y divido entre 1000; la operación inversa que deja la estructura en su tamaño original se hace fuera de la medición. En `pushBack`, `popBack`, `topBack` y `find` el lote se reduce a medida que crece n ($2\,000\,000 / n$, entre 1 y 1000) para que los métodos $O(n)$ no se demoren demasiado, y cada medición se repite 50 veces y se promedia. En `erase`, `addBefore`, `addAfter` y `delete` el objetivo se elige al azar, con semilla fija en 42 para que sea reproducible, y cada operación se mide por separado, con 100 repeticiones como máximo (5 como mínimo). En `erase/addBefore/addAfter`, si el valor al azar no se encuentra en la lista (puede pasar porque ya se borró antes), esa repetición se descarta y el

promedio se divide entre las que sí se midieron. Los valores guardados son los enteros de 0 a n-1 y lo aleatorio es el objetivo de la búsqueda.

Los tamaños fueron $n = 10, 100, 1000, 10\,000, 100\,000$ y $1\,000\,000$. El enunciado menciona hasta 10^8 como ejemplo, pero una sola corrida de pushBack en SinglyNoTail con $n=10^6$ ya tarda varios milisegundos por operación, y con 10^8 el laboratorio no terminaba en un tiempo razonable (y 100 millones de nodos tampoco caben bien en memoria). Con 10^6 ya se ve claro en las gráficas la tendencia: una recta que sube en log-log para $O(n)$ y una línea plana para $O(1)$; subir más solo iba a aumentar el tiempo de espera, no a cambiar la conclusión.

Antes de empezar a medir, cada clase de benchmark corre un calentamiento de la JVM: ejecuta miles de operaciones de cada método sobre una estructura chica, sin registrar nada, para que el JIT alcance a compilar esos métodos a código nativo antes de la medición real. Sin esto, las primeras repeticiones de cada corrida salen artificialmente lentas y contaminan el promedio, sobre todo en los tamaños con pocas repeticiones.

Ojo con las repeticiones: en erase, addBefore, addAfter y delete, que son más costosos, las mediciones individuales son pocas (por ejemplo 5 en $n=10$ y $n=100$), así que con tamaños chicos el promedio queda más expuesto al ruido: una pausa del recolector de basura o del sistema pesa más sobre el promedio. Antes de cada bloque de mediciones llamo a System.gc() para que una recolección no caiga justo dentro del tiempo medido.

4. Complejidad Teórica

4.1 List: Complejidad Teórica

método	SinglyNoTail	SinglyWithTail	DoublyNoTail	DoublyWithTail
pushFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
pushBack	$O(n)$	$O(1)$	$O(n)$	$O(1)$
popFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
popBack	$O(n)$	$O(n)$	$O(n)$	$O(1)$
topFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
topBack	$O(n)$	$O(1)$	$O(n)$	$O(1)$
find	$O(n)$	$O(n)$	$O(n)$	$O(n)$
erase	$O(n)$	$O(n)$	$O(1)$	$O(1)$
addBefore	$O(n)$	$O(n)$	$O(1)$	$O(1)$
addAfter	$O(1)$	$O(1)$	$O(1)$	$O(1)$

4.2 MyStack y MyQueue: Complejidad Teórica

operación	Arreglo dinámico	Arreglo circular
push / enqueue	$O(1)$ amortizado	$O(1)$ amortizado
pop (Stack)	$O(1)$	$O(1)$
dequeue (Queue)	$O(n)$	$O(1)$
peek / front	$O(1)$	$O(1)$
delete(valor)	$O(n)$	$O(n)$

5. Hipótesis

Hipótesis sobre List

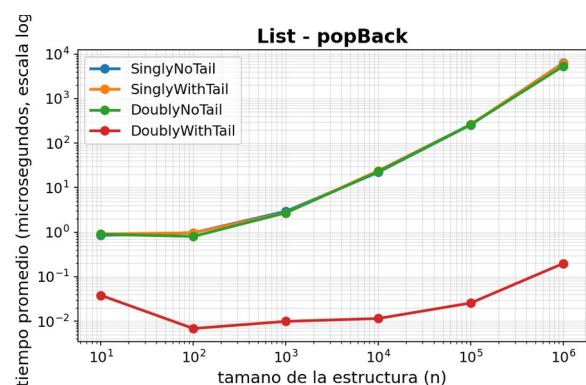
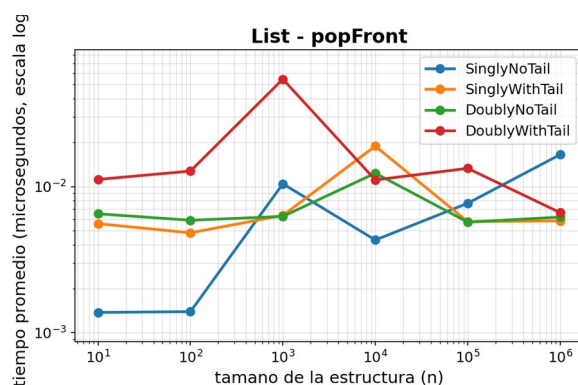
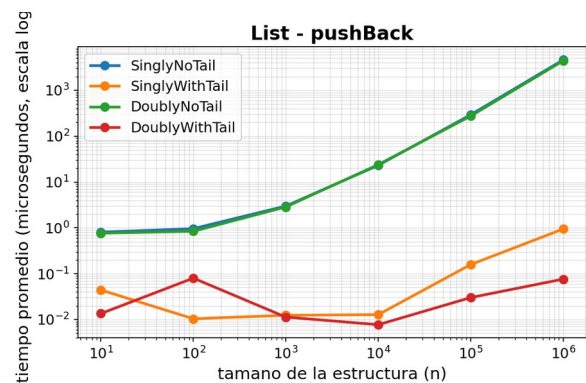
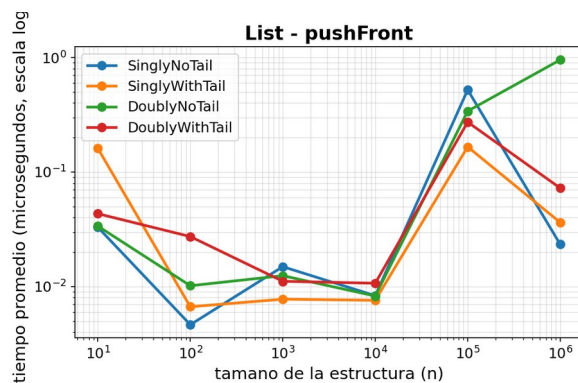
Antes de correr el benchmark, con solo mirar la tabla de arriba ya me imaginaba qué iba a pasar: pushFront, popFront, topFront y addAfter deberían mantenerse casi planos ($O(1)$) en las 4 variantes por igual; pushBack, popBack y topBack solo rápidos en las versiones con tail (o con prev, para popBack en DoublyWithTail), y creciendo en las demás; y find lento ($O(n)$) en las 4, porque ninguna variante indexa directo. Ahora toca ver qué tanto le atiné comparando esto con lo medido.

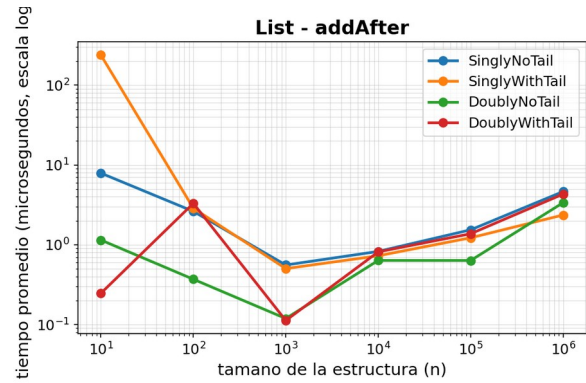
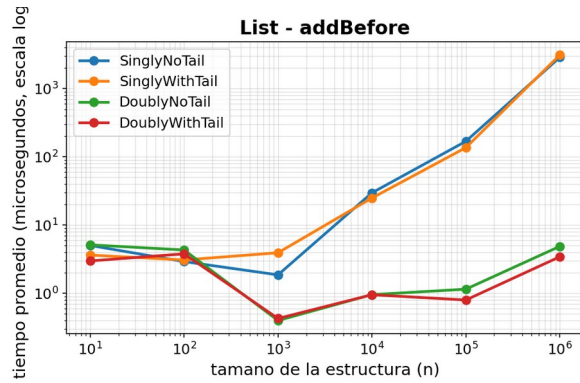
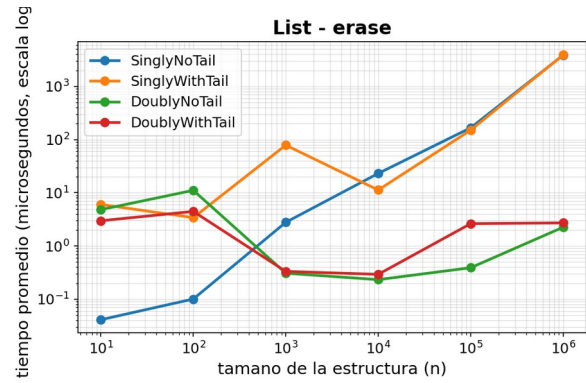
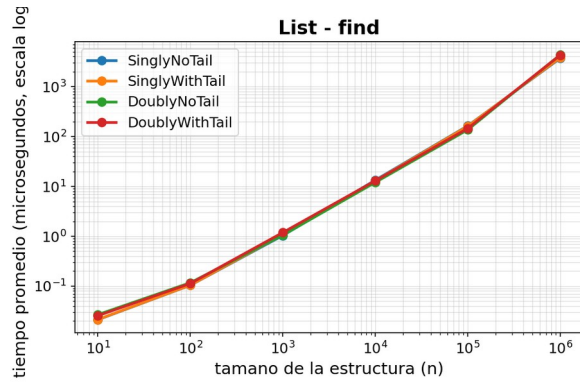
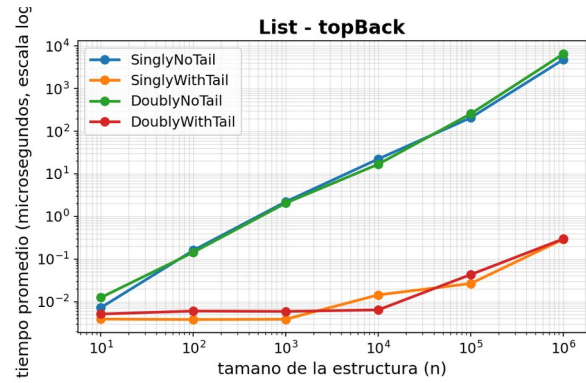
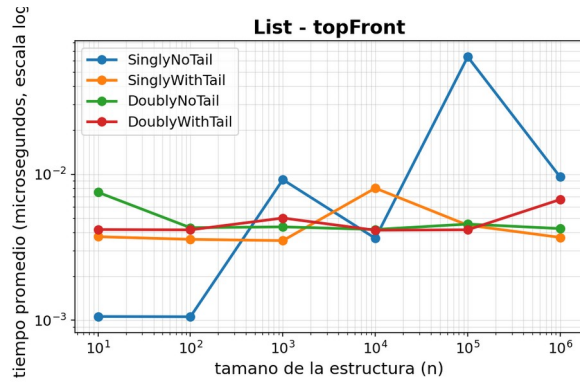
Hipótesis sobre MyStack y MyQueue

Acá la hipótesis era más sencilla: esperaba que push, pop y peek dieran prácticamente igual entre el arreglo dinámico y el circular, porque los dos trabajan sobre el mismo extremo y son $O(1)$. Donde sí esperaba diferencia clara era en dequeue de Queue: $O(n)$ en el dinámico, porque hay que correr todo el arreglo al sacar por el frente, contra $O(1)$ en el circular, donde solo se mueve el índice. En el circular también esperaba que cada operación saliera un poco más cara por el cálculo del módulo. Y esperaba que el push siguiera siendo $O(1)$ amortizado aunque el arreglo se redimensione, por eso lo medí aparte armando la pila desde vacía.

6. Resultados

6.1 List: Resultados Empíricos por Método





SinglyLinkedListNoTail

n	10	100	1000	10000	100000	1000000
pushFront	0.033	0.005	0.015	0.008	0.521	0.023
pushBack	0.804	0.961	2.98	23.05	292.58	4593.63
popFront	0.001	0.001	0.010	0.004	0.008	0.017
popBack	0.848	0.969	2.94	22.13	258.36	6310.24
topFront	0.001	0.001	0.009	0.004	0.064	0.010
topBack	0.007	0.160	2.22	22.32	207.71	4795.31
find	0.021	0.104	1.03	13.32	163.20	3692.34
erase	0.041	0.100	2.78	23.11	166.59	3871.64
addBefore	5.02	2.91	1.86	29.67	169.10	2875.49
addAfter	7.90	2.62	0.557	0.824	1.54	4.64

SinglyLinkedListWithTail

n	10	100	1000	10000	100000	1000000
pushFront	0.161	0.007	0.008	0.008	0.166	0.036
pushBack	0.044	0.010	0.012	0.013	0.158	0.942
popFront	0.006	0.005	0.006	0.019	0.006	0.006
popBack	0.915	0.966	2.73	24.03	263.12	6185.29
topFront	0.004	0.004	0.004	0.008	0.004	0.004
topBack	0.004	0.004	0.004	0.014	0.027	0.290
find	0.021	0.105	1.07	12.92	160.59	3789.47
erase	6.03	3.41	78.60	11.26	150.76	3915.11
addBefore	3.63	3.08	3.93	24.91	136.50	3113.85
addAfter	238.62	2.87	0.502	0.726	1.22	2.36

DoublyLinkedListNoTail

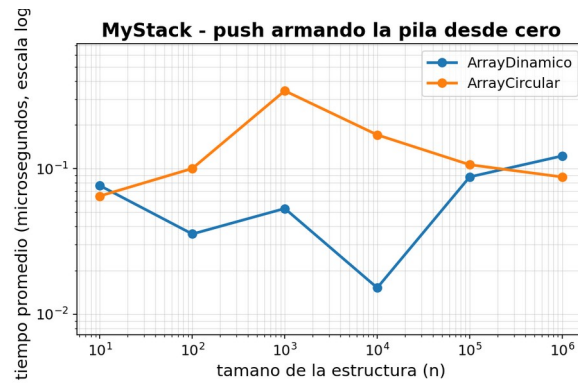
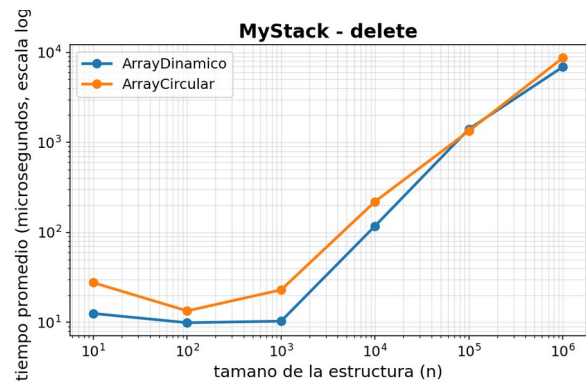
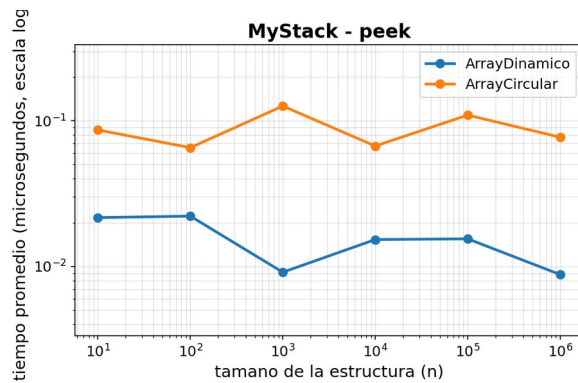
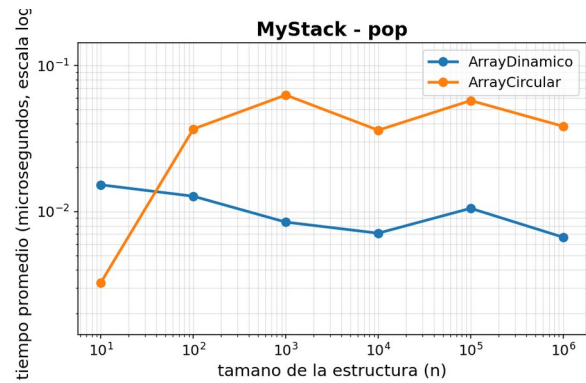
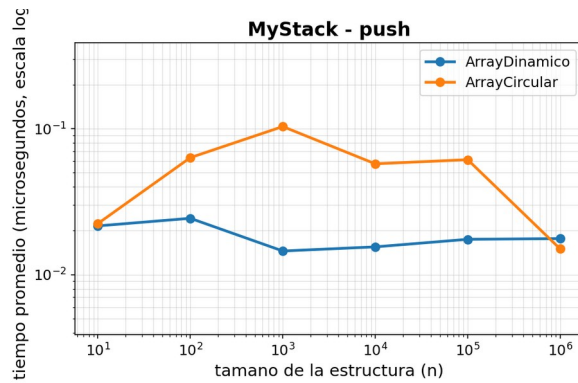
n	10	100	1000	10000	100000	1000000
pushFront	0.034	0.010	0.012	0.008	0.340	0.953
pushBack	0.767	0.841	2.85	23.91	274.22	4387.53
popFront	0.007	0.006	0.006	0.012	0.006	0.006
popBack	0.901	0.801	2.72	22.80	261.52	5345.64
topFront	0.008	0.004	0.004	0.004	0.005	0.004
topBack	0.012	0.144	2.06	16.81	257.11	6518.70
find	0.027	0.117	1.05	12.03	136.46	4328.37
erase	4.82	11.10	0.307	0.232	0.389	2.25
addBefore	5.13	4.31	0.398	0.954	1.15	4.87
addAfter	1.15	0.369	0.119	0.635	0.634	3.37

DoublyLinkedListWithTail

n	10	100	1000	10000	100000	1000000
pushFront	0.043	0.027	0.011	0.011	0.273	0.072
pushBack	0.013	0.080	0.011	0.008	0.030	0.076
popFront	0.011	0.013	0.054	0.011	0.013	0.007
popBack	0.038	0.007	0.010	0.011	0.026	0.198
topFront	0.004	0.004	0.005	0.004	0.004	0.007
topBack	0.005	0.006	0.006	0.006	0.043	0.296
find	0.026	0.113	1.19	12.94	144.86	4284.60
erase	2.98	4.46	0.332	0.292	2.62	2.72
addBefore	2.98	3.78	0.426	0.951	0.796	3.43
addAfter	0.246	3.30	0.112	0.813	1.37	4.31

Comparando esto con la hipótesis: en general sí se cumplió el patrón esperado, sobre todo en n grande (10^5 y 10^6), donde las curvas $O(n)$ (pushBack y topBack sin tail, popBack sin prev, find, erase/addBefore sin prev) se separan bastante de las que quedan planas. topFront en particular se mantiene por debajo de $0.2 \mu s$ en las 4 variantes incluso en $n=1\ 000\ 000$, confirmando que no depende de tail ni de prev. Donde no coincidió tan limpio fue en algunos tamaños chicos e intermedios: varios métodos en teoría $O(1)$ muestran picos puntuales que no tienen que ver con la complejidad sino con ruido del sistema (aun con el calentamiento de JVM, una pausa del recolector de basura pesa más cuando lo que se está midiendo son solo unos pocos nanosegundos por operación).

6.2 MyStack: Resultados Empíricos (Dinámico vs Circular)



Arreglo Dinámico

n	10	100	1000	10000	100000	1000000
push	0.022	0.024	0.015	0.016	0.017	0.018
pop	0.015	0.013	0.008	0.007	0.011	0.007
peek	0.022	0.022	0.009	0.015	0.015	0.009
delete	12.54	9.90	10.30	117.48	1405.81	6893.87
push desde 0	0.076	0.036	0.053	0.015	0.088	0.122

Arreglo Circular

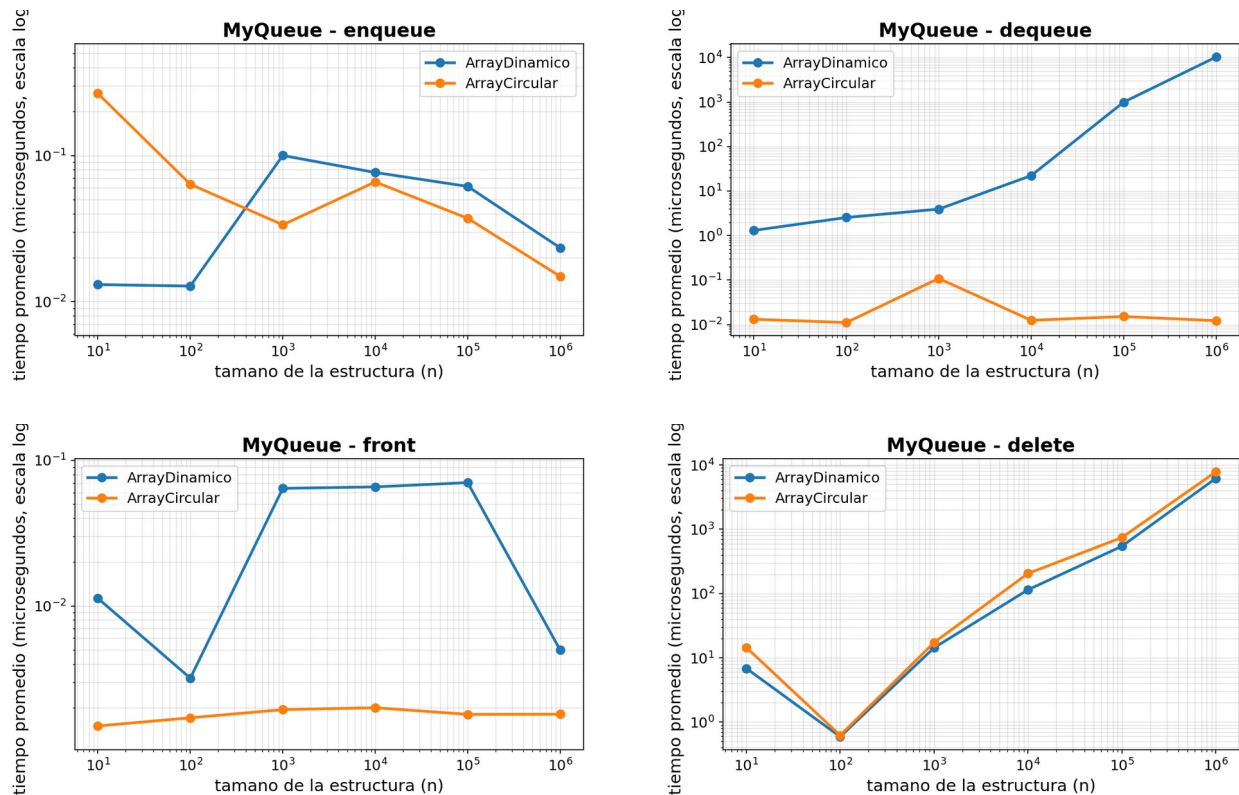
n	10	100	1000	10000	100000	1000000
push	0.022	0.063	0.103	0.057	0.061	0.015
pop	0.003	0.037	0.063	0.036	0.058	0.038

peek	0.086	0.065	0.126	0.067	0.109	0.077
delete	27.61	13.40	22.95	219.69	1338.25	8685.72
push desde 0	0.065	0.100	0.342	0.170	0.106	0.088

Comparando con la hipótesis: en push, pop y peek las dos versiones se mantienen bajas y sin tendencia de crecimiento con n, como se esperaba de operaciones $O(1)$ (por ejemplo push en el dinámico va de 0.022 μs en $n=10$ a 0.018 μs en $n=1\,000\,000$). La versión circular sale más lenta en las tres, de forma bastante constante (por ejemplo pop de 0.038 μs contra 0.007 μs y peek de 0.077 μs contra 0.009 μs en $n=1\,000\,000$), que es el costo del módulo en cada acceso. Sigue siendo una diferencia de centésimas de microsegundo y no crece con n, pero en el Stack el circular no aporta ninguna ventaja, porque igual solo se opera por el tope. delete sí crece claramente con n en las dos, hasta 6893.87 μs (dinámico) y 8685.72 μs (circular) en $n=1\,000\,000$, coherente con el $O(n)$ de buscar más correr el arreglo.

La fila "push desde 0" es el tiempo promedio por push armando la pila desde vacía hasta n elementos, o sea contando los redimensionamientos. Se queda alrededor de 0.122 μs en el dinámico y 0.088 μs en el circular con $n=1\,000\,000$, sin crecer con n, que es lo que predice el análisis amortizado de la sección 2: duplicar la capacidad hace que el costo de copiar se reparta entre todas las inserciones.

6.3 MyQueue: Resultados Empíricos (Dinámico vs Circular)



Arreglo Dinámico

n	10	100	1000	10000	100000	1000000
enqueue	0.013	0.013	0.100	0.077	0.061	0.023
dequeue	1.30	2.54	3.93	22.29	999.97	10319.48
front	0.011	0.003	0.064	0.066	0.070	0.005

delete	6.87	0.593	14.34	114.53	544.63	6193.14
---------------	------	-------	-------	--------	--------	---------

Arreglo Circular

n	10	100	1000	10000	100000	1000000
enqueue	0.266	0.064	0.034	0.066	0.037	0.015
dequeue	0.013	0.011	0.109	0.012	0.015	0.012
front	0.001	0.002	0.002	0.002	0.002	0.002
delete	14.55	0.623	17.33	205.29	744.69	7812.29

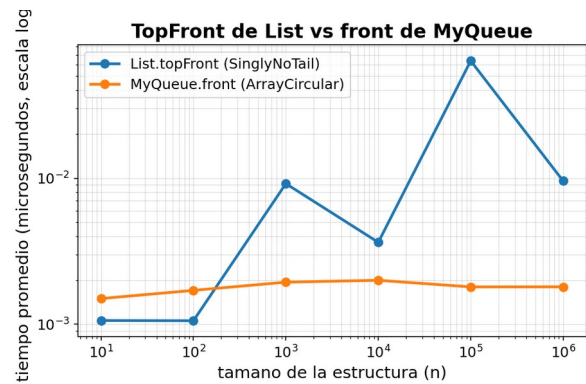
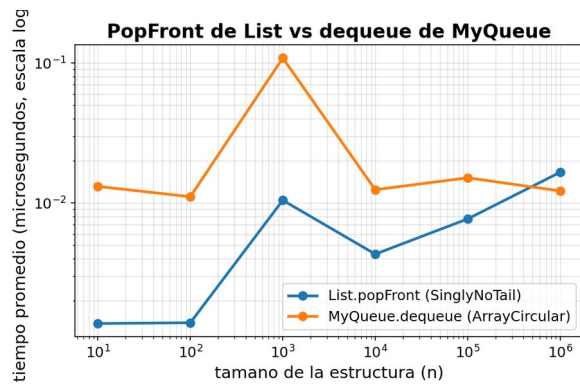
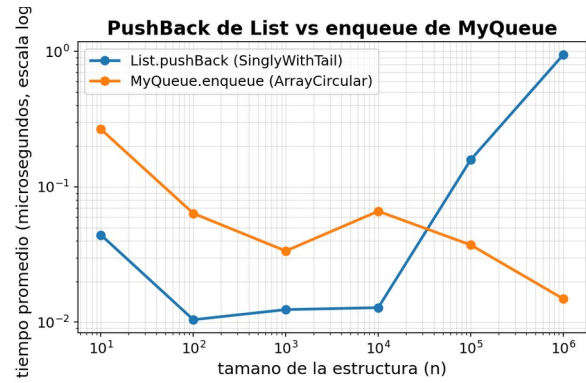
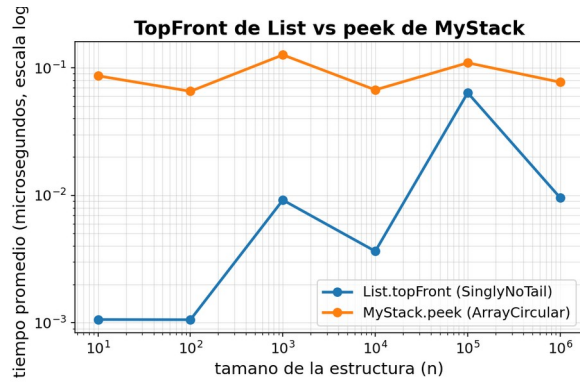
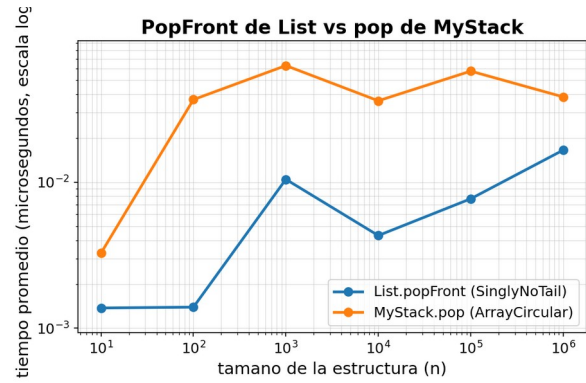
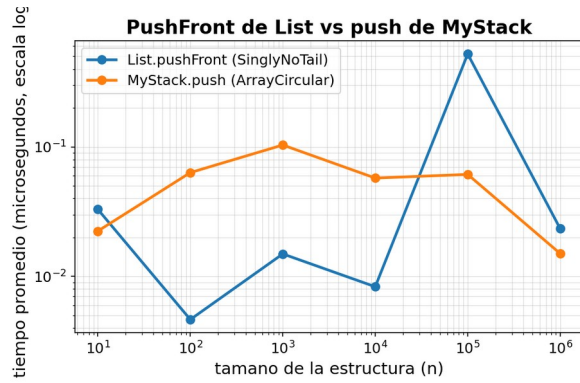
Comparando con la hipótesis: en enqueue y front las dos implementaciones se mantienen bajas y sin tendencia clara de crecimiento, como se esperaba de operaciones $O(1)$. La diferencia que predecía la teoría sí se ve en dequeue: el arreglo dinámico pasa de 1.30 μs en $n=10$ a 10319.48 μs en $n=1\ 000\ 000$, mientras el circular se queda en 0.012 μs , justo la brecha entre $O(n)$ y $O(1)$ de la tabla teórica. delete sigue siendo $O(n)$ en las dos (6193.14 μs en el dinámico y 7812.29 μs en el circular con $n=1\ 000\ 000$), porque en ambos hay que buscar el valor y cerrar el espacio, así que el circular no ayuda con esto.

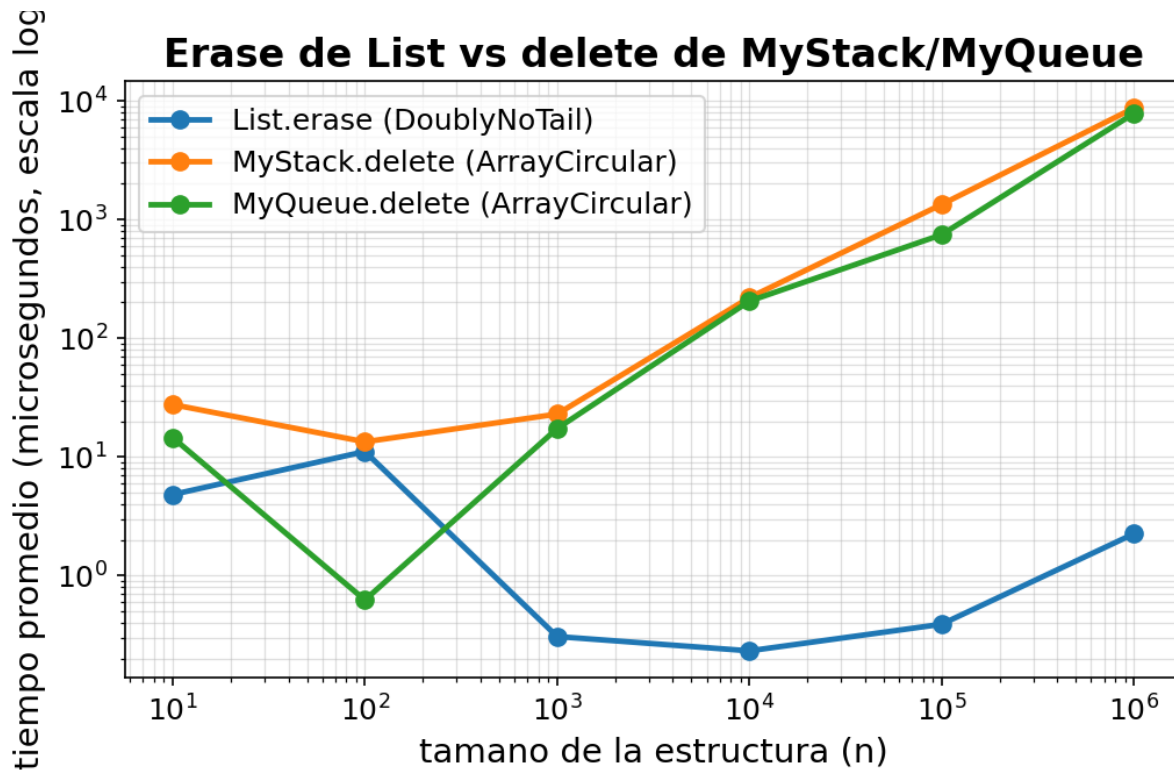
7. Comparación entre Implementaciones

Comparativa de Métodos Equivalentes (List vs MyStack/MyQueue)

El enunciado pide comparar cada método de List contra la implementación de arreglo circular de MyStack y MyQueue, eligiendo para cada método la implementación de List que dé la complejidad más óptima -no tiene que ser siempre la misma variante-, y explicando por qué. Por eso en esta sección uso siempre la versión circular de Stack/Queue. Push/pop de MyStack equivalen a pushFront/popFront de List (ambos operan sobre un mismo extremo), mientras que enqueue/dequeue de MyQueue equivalen a pushBack/popFront (se inserta por un extremo y se saca por el otro). Con eso en mente, elegí así:

- **PushFront vs push, PopFront vs pop/dequeue, TopFront vs peek/front:** estos métodos son $O(1)$ por igual en las 4 variantes de List (topFront/pushFront/popFront solo leen o modifican head, no dependen de tail ni de prev), así que no tenía sentido usar la variante más pesada: elegí SinglyLinkedListNoTail, la más simple que ya da la misma complejidad óptima.
- **PushBack vs enqueue:** pushBack es $O(1)$ solo en las variantes con tail. Entre esas dos, elegí SinglyLinkedListWithTail en vez de DoublyLinkedListWithTail, porque el puntero prev de la versión doble no aporta nada extra para este método en particular.
- **Erase vs delete (Stack y Queue):** erase es $O(1)$ solo en las variantes Doubly (necesitan prev para no tener que recorrer buscando el nodo anterior). Entre esas dos elegí DoublyLinkedListNoTail, porque el puntero tail de la versión con cola no ayuda en nada para borrar un nodo interno.





- **Push/Pop/Peek/Enqueue/Dequeue/Front:** en las seis comparaciones las estructuras quedan en el mismo orden de magnitud, todas $O(1)$; las diferencias entre ellas son de centésimas de microsegundo y se mueven con el ruido de la medición. Con el arreglo dinámico plano, en cambio, la comparación de dequeue contra List.popFront sí hubiera mostrado una brecha enorme en n grande (sección 6.3).
- **Delete / Erase:** acá sí hay diferencia real: en eliminación por valor, List le gana a Stack/Queue basados en arreglo. Las tres son $O(n)$ para encontrar el valor, eso no cambia, pero en la lista enlazada, una vez encontrado el nodo, eliminarlo es $O(1)$; en un arreglo (circular o no) toca correr los elementos que quedan para cerrar el espacio.

8. Conclusiones

- **Listas Enlazadas:** ninguna implementación de List gana en todo. SinglyLinkedListNoTail es la más fácil de programar, pero no escala en las operaciones que tocan el final: pushBack pasa de 0.804 μs en $n=10$ a 4593.63 μs en $n=1\,000\,000$. Si lo que se busca es eficiencia $O(1)$ en ambos extremos (incluyendo topBack), no tiene vuelta de hoja: toca tener doble enlace y puntero a tail al mismo tiempo, o sea DoublyLinkedListWithTail.
- **Arreglo Circular vs Dinámico:** el arreglo circular deja mucho mejor parada a la Cola: dequeue en ArrayDinamico llega a 10319.48 μs en $n=1\,000\,000$, contra 0.012 μs en ArrayCircular. En el Stack la diferencia es mínima, y si algo el circular sale un poco más lento por el módulo, porque ahí las dos versiones ya operaban en $O(1)$ sobre el tope y el circular no tenía nada que mejorar.
- **Uso en escenarios reales:** una pila (Stack) es la estructura natural cuando el orden de deshacer importa: la pila de llamadas de cualquier programa, el Ctrl+Z de un editor, o evaluar expresiones matemáticas con paréntesis. Una cola (Queue) encaja mejor cuando hay que procesar las cosas en el mismo orden en que llegaron: una cola de impresión, los mensajes de un sistema de colas (RabbitMQ,

SQS), o el recorrido BFS de un grafo. En ambos casos, si el número máximo de elementos es predecible, un arreglo (dinámico o circular) gasta menos memoria; si no se sabe de antemano o se necesita insertar/borrar en medio, una lista enlazada da más flexibilidad sin pagar el costo de mover memoria.

- **Arreglos vs Listas:** los arreglos siguen ganando en localidad de memoria y costo por elemento sobre extremos fijos, y con el arreglo circular la cola ya no tiene ninguna desventaja de fondo frente a la lista en las operaciones básicas. La única diferencia real que queda es delete/erase: la lista se mantiene por debajo de un par de microsegundos incluso en $n=1\,000\,000$, mientras que delete en las estructuras de arreglo sube a varios miles de microsegundos, porque además de encontrar el valor ($O(n)$ en las tres) el arreglo tiene que correr los elementos para cerrar el espacio. Si lo que se necesita es insertar o eliminar seguido en medio de la estructura, ahí las listas enlazadas siguen siendo la mejor opción.

9. Uso de Inteligencia Artificial

En este trabajo usé inteligencia artificial, específicamente Claude (Anthropic). La usé para organizar el documento final y la estructura del repositorio en GitHub, y para revisar todo el repositorio contra el enunciado: me ayudó a detectar y corregir cosas como el promedio mal calculado en algunas mediciones, la falta de calentamiento de la JVM, las comparativas que faltaban en la parte 3, y a replantear la medición de las operaciones $O(1)$ en lotes para reducir el ruido. También me ayudó a reescribir las implementaciones circulares de Stack y Queue, a escribir los benchmarks y el script de graficación, y a redactar partes de este informe. Las cuatro implementaciones de List y la idea general de las dos estrategias de arreglo son mías, y las decisiones de qué medir y cómo interpretarlo también. Los benchmarks los corrí yo y las gráficas salen de los csv que generan. Todo el contenido generado con ayuda de IA fue revisado y editado por mí para que la información fuera precisa.